ترتيب الإدخال
(Insertion Sort)

كيف نرتب البيانات خطوة بخطوة كما نرتب أوراق اللعب؟

إعداد: الأستاذ محمود اغبارية

↔ الأساسيات: النسخ والإزاحة (Shift Right)

ملاحظة هامة: في البرمجة نحن لا "ننقل" القيمة فنترك مكانها فارغاً، بل **ننسخها (Copy)** ونكتب فوق الخانة المجاورة.

لدينا مصفوفة، نريد إزاحة العناصر من الفهرس 1 إلى 3 لنكتب في الفهرس 1.

10	20	30	40	?
0	1	2	3	4

```
// إزاحة العناصر من الفهرس 1 حتى 3 خطوة لليمين
for (int j = 3; j ⩾ 1; j--) {
    arr[j + 1] = arr[j];
}
```

القاعدة الذهبية: عند الإزاحة لليمين، **نبدأ دائماً من الأخير** لتجنب طمس البيانات. الخانة الأصلية تحتفظ بنسخة من الرقم حتى نكتب فوقها!

تشبيه من الواقع: أوراق اللعب (الشدة)

تخيل أن بيدك اليسرى أوراق مرتبة. يسحب لك صديقك ورقة جديدة بيدك اليمنى. كيف تدخلها في مكانها؟

1. تمسك الورقة الجديدة وتقارنها بالأوراق المرتبة (من اليمين لليسار).
2. تقوم بـ **إزاحة (نسخ)** الأوراق الأكبر منها خطوة لليمين لتفسح المجال.
3. عندما تجد ورقة أصغر منها، تتوقف وتقوم بـ **الكتابة فوق** الخانة المطلوبة بالرقم الجديد.

> هذا بالضبط ما يفعله الحاسوب! يقسم المصفوفة ذهنياً إلى **قسم أيسر مرتب** و **قسم أيمن غير مرتب**.

خطوات الخوارزمية برمجياً

لتطبيق فكرة أوراق اللعب في المصفوفات، نقوم بالتالي:

- نعتبر أن **العنصر الأول** (الفهرس 0) مرتب بحد ذاته.
- نبدأ حلقة (Loop) تمر على باقي العناصر ابتداءً من **الفهرس 1**.
- في كل دورة، **ننسخ** العنصر الحالي إلى متغير جانبي يُسمى **key** (لحمايته من الطمس).
- نقارن الـ **key** بالعناصر التي تسبقه واحدًا تلو الآخر.
- طالما العنصر السابق **أكبر** من الـ **key**، ننسخه خطوة لليمين.
- عندما نجد مكانًا مناسبًا، نضع الـ **key** فيه (نكتبه فوق القيمة الموجودة).

محاكاة بصرية للخوارزمية

نعتبر العنصر الأول (7) مرتباً بحد ذاته (مربع أخضر خفيف).

كود C# الكامل

```
for (int i = 1; i < arr.Length; i++) {
    int key = arr[i]; // ننسخ العنصر للمتغير لكي لا يُطمس
    int j = i - 1;    // j يؤشر على العناصر المرتبة السابقة

    // حلقة الإزاحة: طالما لم نصل للبداية والعنصر السابق أكبر من الـ key
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j]; // ننسخ العنصر لليمين
        j--;                 // نعود خطوة للخلف للمقارنة مع العنصر الذي يسبقه
    }

    // نكتب المتغير فوق القيمة الموجودة، بعد خروجنا من حلقة while
    arr[j + 1] = key;
}
```

ركز جيداً على هذا الكود، فالأسئلة القادمة ستختبر فهمك العميق له!

تمرين 1: متابعة (מעקב)

معطاة المصفوفة التالية:

2	6	4	8

سؤال: كيف ستبدو المصفوفة 2 = i بعد انتهاء الدورة التي يكون فيها تماماً؟

فكر في الإجابة، ثم انتقل للشريحة التالية لمعرفة الحل.

إجابة تمرين 1

الإجابة الصحيحة هي: { 4, 6, 8, 2 }

الشرح خطوة بخطوة:

- **قبل البداية (i = 0):** { 8, 4, 6, 2 }
- **عندما i = 1 (الرقم 4):** ننسخ الـ 8 لليمين ونكتب الـ 4 مكانها.
  تصبح: { 4, 8, 6, 2 }
- **عندما i = 2 (الرقم 6):** نقارنه مع 8 (أكبر، ننسخه لليمين)، ثم مع 4 (أصغر، نتوقف). ونكتب الـ 6 في الفراغ.
  تصبح: { 4, 6, 8, 2 }

❓ تمرين 2: حالة حافة (شرط التوقف)

في حلقة الـ while نكتب الشرط التالي:

```
while (j >= 0 && arr[j] > key)
```

سؤال: ماذا يحدث للبرنامج إذا نسينا كتابة `j >= 0` وتركنا الشرط `arr[j] > key` فقط؟

تلميح: ماذا يحدث عندما يكون الـ key هو أصغر رقم في المصفوفة؟

✅ إجابة تمرين 2

خطأ وقت التشغيل (IndexOutOfRangeException)! 💥

التفسير:

إذا كان العنصر الذي نفحصه (الـ key) هو أصغر عنصر، فإن حلقة الـ while ستستمر في فحص العناصر وتُنقص قيمة `j` حتى تصبح `-1`.

عندما يحاول الحاسوب فحص الشرط `arr[-1] > key` سيؤدي ذلك إلى انهيار البرنامج لأنه لا يوجد فهرس سالب في المصفوفات المعتادة.

> الشرط `j ≥ 0` هو "صمام الأمان" الذي يوقف الحلقة عند الوصول لبداية المصفوفة.

تمرين 3: كفاءة الخوارزمية

لنفترض أننا أعطينا الكود مصفوفة **مرتبة مسبقاً** لتصنيفها، مثل:

1	2	3	4	5

سؤال: كم مرة سيتم تنفيذ الأوامر داخل حلقة الـ while (أوامر النسخ/الإزاحة)؟ ولماذا؟

إجابة تمرين 3

لن يتم الدخول لحلقة الـ while أبداً! (0 مرات)

التفسير:

في كل دورة `i`، سيتم مقارنة الـ key مع العنصر الذي يسبقه مباشرة.

بما أن المصفوفة مرتبة، فالعنصر السابق سيكون دائماً **أصغر** من الـ key، وبالتالي الشرط `arr[j] > key` سيُعطي **False** من الفحص الأول.

> **معلومة هامة:** حالة المصفوفة المرتبة تعتبر "أفضل حالة" (Best Case) لخوارزمية Insertion Sort، وتكون كفاءتها ممتازة جداً $$O(n)$$.

🏆 تمرين 4: تحدي البرمجة

الكود الأساسي الذي تعلمناه يقوم بترتيب المصفوفة **تصاعدياً** (من الأصغر للأكبر).

سؤال: ما هو التعديل الوحيد الذي يجب إجراؤه في شرط الـ while لكي ترتب الخوارزمية المصفوفة تنازلياً (من الأكبر للأصغر)؟

```
while (j ≥ 0 && arr[j] ??? key)
```

إجابة تمرين 4

نغير إشارة الأكبر > إلى أصغر <

```
while (j ⩾ 0 && arr[j] < key)
```

التفسير:

في الترتيب التصاعدي: كنا نزيح العناصر التي هي **أكبر** من المتغير الجديد لليمين.

في الترتيب التنازلي: نريد أن تكون العناصر الكبيرة في البداية، لذلك نزيح العناصر **الأصغر** من المتغير الجديد لليمين لإفساح المجال له.

نهاية الدرس

كل التوفيق والنجاح!

تذكر دائماً: البرمجة تحتاج لورقة وقلم وتتبع الكود خطوة بخطوة.